Making Quantum Computing Open: Lessons from Open Source Projects

Ruslan Shaydulin, Caleb Thomas, and Paige Rodeghero
{rshaydu,caleb8,prodegh}@clemson.edu
School of Computing, Clemson University
Clemson, South Carolina, USA

ABSTRACT

Quantum computing (QC) is an emerging computing paradigm with the potential to revolutionize the field of computing. QC is a field that is quickly developing globally and has high barriers of entry. In this paper, we explore both successful contributors to the field as well as the wider QC community with the goal of understanding the backgrounds and training that helped them succeed. We gather data on 148 contributors to open source quantum computing projects hosted on GitHub and survey 46 members of QC community. Our findings show that QC practitioners and enthusiasts have diverse backgrounds, with most of them having a PhD and training in physics or computer science. We observe a lack of educational resources on quantum computing. Our goal is to use these results to start a conversation on making quantum computing more open.

CCS CONCEPTS

• **Software and its engineering → Open source model**.

KEYWORDS

quantum computing, open source software, contributors

ACM Reference Format:
Ruslan Shaydulin, Caleb Thomas, and Paige Rodeghero. 2020. Making Quantum Computing Open: Lessons from Open Source Projects. In *IEEE/ACM 42nd International Conference on Software Engineering Workshops (ICSEW'20), May 23–29, 2020, Seoul, Republic of Korea.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3387940.3391471

1 INTRODUCTION

The recent years saw a sequence of rapid scientific and engineering advancements in the field of quantum computing (QC). In less than two decades, QC evolved from being mostly theoretical to being able to solve practically interesting problems [14, 29, 30, 33]. A number of private companies, universities, and government labs worldwide successfully demonstrated a hardware implementation of QC [8–10, 24, 28], and a number of quantum computing projects have emerged, aiming to provide a platform for integration of quantum computation in scientific and business workflows.

Quantum computing has the potential to provide exponential speedups to many important problems [22], and has implications for RSA-based cybersecurity [22]. Many more algorithms have been developed, with applications to quantum chemistry [27], combinatorial optimization [13, 14] and machine learning [30].

To fully realize the promise of quantum computing, the community needs scientists and developers with skills relevant to the evolving field. In these early days, researchers working in quantum computing need an understanding of both the unique logic of quantum computation, as well as the computer science experience needed to integrate it into existing classical workflows. One example of an area where a diverse set of skills spanning multiple fields is required is the development of quantum algorithms. Quantum algorithms like Shor's [31] are significantly different from their classical counterparts. Just understanding them requires knowledge of quantum mechanics and computational complexity theory, two disciplines that usually do not go together in a university curriculum. The development of new algorithms requires mastery of both, as well as a deep computer science background.

Our data shows that currently, most QC practitioners deal with the problem of lack of educational resources by combining their formal training in fields like physics or computer science with reading papers and textbooks on their own and following tutorials online. This makes the field of QC hard to enter and hard to navigate for the newcomers. We address this problem by performing an analysis of the background and skills that practitioners find essential to their work.

In this paper, we present the data about the background and training of QC researchers and practitioners. The methodology that we follow consists of two parts, with possible overlap between the two. First, we scrape the data about 148 contributors to open source QC projects from GitHub. Second, we survey 46 QC professionals to augment the data. We found that most QC specialists are trained in non-computing fields like physics, chemistry, or mathematics. Our goal for this paper is to receive community feedback on this line of research, which could include further investigation of factors contributing to success in QC field through a surveying of practitioners, panels at conferences, and gathering data on open source contributions. We believe our findings can help develop educational resources targeted at preparing a new generation of of QC researchers and practitioners.

After the completion of this work, we became aware of a related work appearing in Ref. [15]. They provide an exhaustive review of open source quantum computing projects. Unlike our work, they do not directly survey the project contributors. Additionally, we review the educational and professional backgrounds of the contributors.

ICSEW'20, May 23–29, 2020, Seoul, Republic of Korea

ACM ISBN 978-1-4503-7963-2/20/05…$15.00
https://doi.org/10.1145/3387940.3391471

2 PROBLEM

Today there are very few training programs in quantum computing, and there is no consensus on the curriculum structure. This is supported by our data: out of 46 survey respondents, only one received formal training in quantum computation. This indicates a lack of understanding of what training is required for success in QC field. Without looking further into what is required to become a part of the QC field, it is difficult for aspiring QC researchers to know what they should do to become a part of this field, which will inevitably stunt the growth of the QC industry. There is a need for a deeper exploration of the challenges facing QC researchers and practitioners. This paper addresses this by exploring the contributors to open source quantum computing projects, their backgrounds, and the challenges they face.

3 BACKGROUND AND RELATED WORK

Quantum computing is still a very young field, with software projects constantly pushing the boundary of human knowledge. As such, quantum computing software projects have to deal with all the problems that plague scientific software projects: unforeseen changes in the requirements, lack of software development expertise and limited budgets [20]. The cross-disciplinary nature of quantum computing adds to the complexity of the domain.

3.1 Quantum Computing

Unlike in classical computation, where the computation happens by manipulating bits, the fundamental computational unit in QC is qubit. A bit can have one of two states: 0 or 1. Similarly, qubit state is a unit vector in a two-dimensional complex vector space [22]. A qubit state can be encoded in a state of a quantum mechanical object, for example as polarization of a single photon [22].

The field of quantum computing is in a state of constant change and is generally expected to continue changing in the foreseeable future. Recently, multiple Near-term Intermediate-Scale Quantum (NISQ) hardware implementations have been developed [26] and demonstrated to provide a potential for quantum speedups [12]. Naturally, different implementations come with certain trade-offs. For example, trapped ion qubits are generally less noisy and offer better connectivity, whereas superconducting qubits offer faster gate clock speeds and more clear path to scalability [21]. This diversity of hardware introduces an additional degree of complexity for the development of QC algorithms and software, forcing algorithm developers to stay aware of the trade-offs presented by hardware.

A plethora of algorithms leveraging the power of quantum computation has been developed over the years. Shor's [31] and Grover's [17] algorithms are the two most well-known examples of quantum algorithms for practical problems with theoretically proven speedups over the classical state-of-the-art. However, the limitations of NISQ-era hardware make most of them impossible to run in the near-term. Near-term quantum computers are widely believed to be able to provide no more than a few hundreds of non-error-corrected qubits. To address this challenge, a number of NISQ approaches have been proposed, most prominent of them Variational Quantum Eigensolver (VQE) [23] and Quantum Approximate Optimization Algorithm (QAOA) [13]. The limitations of near-term hardware make the development of practical algorithms especially challenging.

3.2 Open Source Scientific Software Projects

There has been a push towards open source scientific software projects. Government labs, such as Sandia National Laboratories, create and use open source projects [1, 7]. This allows for easier collaboration with scientists and programmers both within industry and academia.

Many software engineering studies have been conducting by mining GitHub [11, 18, 19, 25, 34]. These projects look at contributors, their efforts, the number of bugs created or fixed, etc. One problem with open source projects is that it can be unclear who is contributing more than others [16]. Despite its limitations, mining collaborative repositories can provide valuable insight into the structure of a given scientific community.

4 METHODOLOGY

In order to understand the structure of the quantum computing community and the challenges facing the contributors, we focus on three open source and open-development quantum computing projects from three companies working in the quantum computing field: Qiskit (IBM), PyQuil/Grove (Rigetti) and Cirq (Google).

Qiskit is open source project developed by IBM. It consists of two main parts: Qiskit Terra [3] provides the basic building blocks and Qiskit Aqua [2] provides a library of algorithms upon which applications can be built. PyQuil/Grove is an open source QC framework developed by Rigetti. Rigetti PyQuil [6] provides the low-level building blocks and Rigetti Grove [5] provides a library of algorithms implemented with PyQuil. Cirq [4] is developed by researchers at Google. Cirq is not an official Google product. Cirq provides low-level tools for building programs to be run on noisy intermediate-scale quantum (NISQ) computers.

Our approach consists of two parts. First, we scraped the contribution data from the projects' GitHub repositories using PyDriller framework [32]. In addition to collecting contribution data, we collected public users handles and email addresses. These were then used to manually connect these users to their professional profiles on personal websites or social networks such as LinkedIn. We were able to connect more than 95% of the harvested emails and handles to publicly available detailed profiles. This data provide us with a quantitative understanding of who contributes to quantum computing projects. QC field is still relatively small when compared with software development as a whole. Analyzing the data on between a hundred and two hundred QC project contributors provides valuable insight into the QC practitioners. Second, we augment the data by sending out a questionnaire to quantum computing developers and enthusiasts through IBM Qiskit and Rigetti Slack channels, as well as to emails of contributors we scraped from GitHub.

The questionnaire extends the data harvested from GitHub by exploring the background, motivation, and the challenges facing the practitioners. The questionnaire consisted of three parts. The first part included questions on the participants' background, education, and professional experience (e.g. "What was your major in college?"). The second part explored participants' experience with different QC frameworks, educational resources they find the most helpful and how confident they were in their computer science and physics knowledge (e.g. "How adequate do you find your CS background for your work in QC (e.g. coding skills, understanding of CS concepts etc)?"). The third part consisted of open-ended questions, e.g. "What kind of CS training would have better prepared you for your work in QC? What CS class you wish you took but didn't?".

5 RESULTS

Following the two-pronged approach outlines in Sec. 4, we collected data about 148 quantum computing researchers by scraping data on contributions from GitHub repositories of popular QC projects and received 46 responses to our survey. The first part of the data (GitHub data) gives us an insight about the people who *already contribute* to quantum computing projects, whereas the second part of the data (survey data) gives us a broader picture of the community of people *interested* in quantum computing. This is due to the fact that GitHub data only contains information about the people who have achieved the level of competency necessary to get their contribution approved and included in the mater of a given repository, whereas the survey data includes a broader slice of people who are interested enough in QC to join a dedicated Slack channel. We observe a significant difference in composition between the two, indicating that QC companies need to engage the broader community in the software development process to make these projects truly collaborative. The difference in composition makes the two parts of the dataset complimentary, as they address different parts of the community. The GitHub data also demonstrates how different companies approach the challenging task of building a quantum computing framework in different ways.

The data we collected is available online at http://shaydul.in/papers/QSE2020-data.zip. We have removed any information that could potentially be used to personally identify the participants.

5.1 Data Analysis

Through analysis of the GitHub data, we get an idea of people who are already successful in the field. Concretely, we can measure the success by a number of commits to the state-of-the-art open source projects. If success is defined like that, the data shows that most successful contributors have a PhD. 61% of contributors with more than 10 commits in repository's master branch have a PhD, with 81% of them having a PhD in physics (55%) or computer science (26%). This is unsurprising since the projects in question are fundamentally research projects, requiring precisely the skills a PhD program develops. The large share of people with PhDs is consistent across companies, with the ratio being 62% in IBM, 64% in Rigetti and 50% in Google's Cirq. All the projects we looked at is that they were all developed almost exclusively by people employed at that company, with the ratio of employees among top contributors being 100% for all three companies. This indicates a need for broader collaboration, both between companies, as well as between companies and other research institutions.

In contrast to the GitHub data, which contains almost exclusively employees of the QC companies, out of 46 survey respondents, only 30% indicated that they work for a quantum-computing-centered company. This shows that there is a broader interest in quantum computing, both in the industry (35% of respondents) as well as in national laboratories (9%) and among PhD students (9%). Still, we observe a very high number of people with PhD among our respondents (28%), confirming our observations from GitHub data. Similarly to GitHub data, most respondents (70%) studied either physics or computer science, though the balance between the two disciplines is different (29% physics and 41% computer science).

The predominance of people with PhDs (i.e. people with research backgrounds) is indicative of a young field, where the barrier to entry is still very high. There is a clear need to lower those barriers by developing educational materials that could help bring people into the field. The lack of educational materials is confirmed by multiple observations. First, our data shows almost no people with a degree in Quantum Computation, Quantum Information or related fields (only one respondent indicated that they have a masters degree in Quantum Computing). Second, we asked respondents to rank educational materials by their impact and helpfulness, with the first being the most helpful and the last being the least helpful. Then for each type of educational material we compute a mean score by calculating the mean ranking of that material across the collected responses. We observe the respondents rate "reading papers on their own" (mean score of 2.98, less is better) and "reading textbooks on their own" (mean score of 3.04) as most helpful, followed by "following tutorials in software repositories" (mean score of 3.25). This shows that the effort software companies have put into developing introductory tutorials to accompany their frameworks is paying off and that the community finds them helpful. Surprisingly, Massive Open Online Courses (MOOCs) scored fairly low (second to last with mean score of 3.91), which in our opinion says more about their accessibility rather than availability (that is how easy they are to follow, rather then how easy they are to find). The only online MOOCs known to us are parts of MIT Quantum Computing Curriculum, require a strong mathematical background, and teach mostly theoretical (and somewhat challenging) material.

The data we collected shows that training for quantum computing should combine physics, computer science (CS) and mathematics. We find that respondents trained in predominantly non-CS fields find their computer science skills lacking (skills like coding, software development etc), whereas respondents with predominantly CS training find their physics knowledge inadequate. Interestingly, the ratio of respondents who found their physics skills moderately or extremely inadequate (20% of respondents) is much higher that the ratio of respondents who said the same about their CS skills (9%). This indicates that the community finds the physics side harder, suggesting that the training should focus more on developing relevant physics knowledge. In an open-ended question, where the respondents were encouraged to discuss what training they wish they received, 17 respondents indicated that they did not receive sufficient training in quantum computing and quantum information. Another commonly received suggestion was linear algebra and mathematical training in general.

5.2 Threats to validity

The two main threats to validity are rapid growth in number of contributors to open-source QC projects and the low response rate to the survey.

First, the number of contributors and the amount of contributions to the repositories explored in this study is growing rapidly. This study has been completed in December 2018, so the data is representative of the state of open-source QC community at the time. However, we had to wait to find an appropriate venue for this work, resulting in it only being submitted in February 2020. This explains the discrepancy in number of contributors, as it has grown from 148 contributors analyzed in this study to 260+ currently.

Second, we observe a low response rate to the survey. At the time, Slack channels that we used to distribute the survey had more than a thousand subscribers. We note that this is partially due to the fact that only a small share of the people who subscribe to a given Slack channel checks it regularly and participates in the discussion. This makes it impossible for us to estimate the true response rate or analyse the distribution of the channels members to estimate how representative the sample was.

With these threats in mind, we believe that sharing these results with the broader quantum software engineering community is beneficial, as they shed light on important and rarely discussed aspects of the software development for quantum computing.

6 DISCUSSION AND CONCLUSION

In this paper, we present the data on 148 contributors to quantum computing projects and 46 members of the wider quantum computing community (with possible overlap between the two). Due to the anonymity of the survey respondents, it is impossible to know how large the overlap is exactly. However, the differences in the results observed in GitHub and survey data indicate that the overlap is not very large. Therefore the two parts of our dataset complement each other, providing valuable insight into those factors contributing to success in the field of quantum computing.

Our finding that most respondents rate studying on their own as the most useful highlights the need for universities and other educational institutions to step in and offer training, be it in the form of a MOOC available on the internet or an in-person degree program. These educational resources should adequately prepare QC practitioners, with more focus given to the most challenging parts of QC curriculum. Our findings have implications for designs of such offerings, indicating that the respondents find computer science and software engineering aspects of their work in QC the least challenging while struggling more with underlying fundamental mathematical and physical concepts.

The majority of the respondents have less than two years of experience in QC (80%), with only 7% having worked in the field for more than 5 years. This is indicative of a young and booming field, with a lot of new entrants. The relative immaturity of the field, as well as its interdisciplinarity, make it very challenging to join. As a community, we need to develop more resources to bringing quantum computing to undergraduate students through classes, online courses, and tutorials. But as the field is booming and the hype is high, it's important to keep a cool head. As one of our respondents wisely noted, one should "be careful not to train people for something that will bust and not get them jobs."

We hope this project will have a positive impact by bringing the software engineering community and the software engineering methods to quantum computing. Moreover, we believe that the data we collected will contribute to the discussion on how best to prepare the next generation of quantum computing specialists and researchers.

REFERENCES

[1] [n.d.]. Federal Source Code Policy: Achieving Efficiency, Transparency, and Innovation through Reusable and Open Source Software. https://sourcecode.cio.gov/. Accessed: 2019-02-03.
[2] [n.d.]. Qiskit/aqua repository. https://github.com/Qiskit/aqua
[3] [n.d.]. Qiskit/qiskit-terra repository. https://github.com/Qiskit/qiskit-terra
[4] [n.d.]. quantumlib/Cirq repository. https://github.com/quantumlib/Cirq
[5] [n.d.]. rigetticomputing/grove repository. https://github.com/rigetticomputing/grove
[6] [n.d.]. rigetticomputing/pyquil repository. https://github.com/rigetticomputing/pyquil
[7] [n.d.]. Software Policy. https://tinyurl.com/yaoyd8kl. Accessed: 2019-02-03.
[8] Antonio Acín, Immanuel Bloch, Harry Buhrman, Tommaso Calarco, Christopher Eichler, Jens Eisert, Daniel Esteve, Nicolas Gisin, Steffen J Glaser, Fedor Jelezko, et al. 2018. The quantum technologies roadmap: a European community view. *New Journal of Physics* 20, 8 (2018), 080201.
[9] CJ Ballance, TP Harty, NM Linke, MA Sepiol, and DM Lucas. 2016. High-fidelity quantum logic gates using trapped-ion hyperfine qubits. *Physical review letters* 117, 6 (2016), 060504.
[10] Rami Barends, Julian Kelly, Anthony Megrant, Andrzej Veitia, Daniel Sank, Evan Jeffrey, Ted C White, Josh Mutus, Austin G Fowler, Brooks Campbell, et al. 2014. Superconducting quantum circuits at the surface code threshold for fault tolerance. *Nature* 508, 7497 (2014), 500.
[11] Valerio Cosentino, Javier Luis, and Jordi Cabot. 2016. Findings from GitHub: methods, datasets and limitations. In *Proceedings of the 13th International Conference on Mining Software Repositories*. ACM, 137–141.
[12] Vedran Dunjko, Yimin Ge, and J Ignacio Cirac. 2018. Computational speedups using small quantum devices. *arXiv preprint arXiv:1807.08970* (2018).
[13] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028* (2014).
[14] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm applied to a bounded occurrence constraint problem. *arXiv preprint arXiv:1412.6062* (2014).
[15] Mark Fingerhuth, Tomáš Babej, and Peter Wittek. 2018. Open source software in quantum computing. *PLOS ONE* 13, 12 (12 2018), 1–28. https://doi.org/10.1371/journal.pone.0208561
[16] Max Goldman, Greg Little, and Robert C. Miller. 2011. Real-time Collaborative Coding in a Web IDE. In *Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology* (Santa Barbara, California, USA) *(UIST '11)*. ACM, New York, NY, USA, 155–164. https://doi.org/10.1145/2047196.2047215
[17] Lov K Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. ACM, 212–219.
[18] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M German, and Daniela Damian. 2014. The promises and perils of mining GitHub. In *Proceedings of the 11th working conference on mining software repositories*. ACM, 92–101.
[19] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M German, and Daniela Damian. 2016. An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering* 21, 5 (2016), 2035–2071.
[20] Catherine Letondal and Uwe Zdun. 2003. Anticipating scientific software evolution as a combined technological and design approach. In *Second International Workshop on Unanticipated Software Evolution*.
[21] Norbert M Linke, Dmitri Maslov, Martin Roetteler, Shantanu Debnath, Caroline Figgatt, Kevin A Landsman, Kenneth Wright, and Christopher Monroe. 2017. Experimental comparison of two quantum computing architectures. *Proceedings of the National Academy of Sciences* 114, 13 (2017), 3305–3310.
[22] Michael A Nielsen and Isaac Chuang. 2002. Quantum computation and quantum information.
[23] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O'brien. 2014. A variational eigenvalue solver on a photonic quantum processor. *Nature communications* 5 (2014), 4213.
[24] Jason R Petta, Alexander Comstock Johnson, Jacob M Taylor, Edward A Laird, Amir Yacoby, Mikhail D Lukin, Charles M Marcus, Micah P Hanson, and Arthur C Gossard. 2005. Coherent manipulation of coupled electron spins in semiconductor quantum dots. *Science* 309, 5744 (2005), 2180–2184.
[25] Daniel Pletea, Bogdan Vasilescu, and Alexander Serebrenik. 2014. Security and emotion: sentiment analysis of security discussions on GitHub. In *Proceedings of the 11th working conference on mining software repositories*. ACM, 348–351.

[26] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *arXiv preprint arXiv:1801.00862* (2018).

[27] Jonathan Romero, Ryan Babbush, Jarrod McClean, Cornelius Hempel, Peter Love, and Alán Aspuru-Guzik. 2018. Strategies for quantum computing molecular energies using the unitary coupled cluster ansatz. *Quantum Science and Technology* (2018).

[28] Mark Saffman, Thad G Walker, and Klaus Mølmer. 2010. Quantum information with Rydberg atoms. *Reviews of Modern Physics* 82, 3 (2010), 2313.

[29] Ruslan Shaydulin, Hayato Ushijima-Mwesigwa, Ilya Safro, Susan Mniszewski, and Yuri Alexeev. 2018. Community Detection Across Emerging Quantum Architectures. *arXiv preprint arXiv:1810.07765* (2018).

[30] Ruslan Shaydulin, Hayato Ushijima-Mwesigwa, Ilya Safro, Susan Mniszewski, and Yuri Alexeev. 2018. Network Community Detection On Small Quantum Computers. *arXiv preprint arXiv:1810.12484* (2018).

[31] Peter W Shor. 1994. Algorithms for quantum computation: Discrete logarithms and factoring. In *Foundations of Computer Science, 1994 Proceedings., 35th Annual Symposium on*. Ieee, 124–134.

[32] Davide Spadini, Mauricio Aniche, and Alberto Bacchelli. 2018. *PyDriller: Python Framework for Mining Software Repositories*. https://doi.org/10.1145/3236024.3264598

[33] Hayato Ushijima-Mwesigwa, Christian FA Negre, and Susan M Mniszewski. 2017. Graph partitioning using quantum annealing on the D-Wave system. In *Proceedings of the Second International Workshop on Post Moores Era Supercomputing*. ACM, 22–29.

[34] Bogdan Vasilescu, Daryl Posnett, Baishakhi Ray, Mark GJ van den Brand, Alexander Serebrenik, Premkumar Devanbu, and Vladimir Filkov. 2015. Gender and tenure diversity in GitHub teams. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. ACM, 3789–3798.